

Application of Model Predictive Path Integral (MPPI) Control to Autonomous Underwater Vehicle (AUV)

Jack Fetkovich

*Electrical and Computer Engineering
North Carolina State University
Raleigh, United States
jjfetkov@ncsu.edu*

Akshay Pradhan

*Electrical and Computer Engineering
North Carolina State University
Raleigh, United States
aspradh4@ncsu.edu*

Abstract—This paper presents an application of Model Predictive Path Integral (MPPI) on a fully actuated autonomous underwater vehicle (AUV) developed by NC State's AquaPack Robotics. Unlike other traditional controllers, MPPI leverages system dynamics and sampling based optimization to improve trajectory tracking while remaining relatively computationally efficient. A simplified model of the AUV is implemented in a popular physics-based simulation, including thrust allocation and generalized hydrodynamics. The controller's performance is evaluated through tracking a realistic competition course trajectory, avoiding obstacles and demonstrating MPPI's potential for reliable, model-based underwater control suitable for deployment and real-time execution.

I. INTRODUCTION

There exists an extensive body of research on trajectory tracking for both ground and aerial vehicles. Precise control of drones is effectively a solved problem, and wheeled ground vehicles are in a similar state. However, ground vehicles only have the capability of traversing around 29% of the Earth's surface. Vehicles capable of maneuvering about the other 71%, which is covered in water, receive significantly less attention. Google Scholar searches for trajectory tracking of "Ground Vehicle", "Aerial Vehicle", and "Underwater Vehicle" return 473,000, 214,000, and 86,900 results respectively. This lack of attention also shows up in the lack of available platforms for simulated experimentation. As will be discussed in detail later, one of the biggest challenges of this project is that most robotic simulation programs are not capable of accurately simulating underwater vehicles out of the box.

The goal of this project is to apply a trajectory tracking algorithm to an autonomous underwater vehicle developed by NC State AquaPack Robotics. Control schemes for previous iterations of the organization's vehicles have been purely error based controllers with no model information included. This was implemented by applying PD controllers on each axis of travel to track velocity setpoints. While this can be effective, the dynamics of underwater vehicles are well enough understood that a superior model-based controller is possible. Model Predictive Control (MPC) is one option, as it incorporates a model of the system to find optimal control

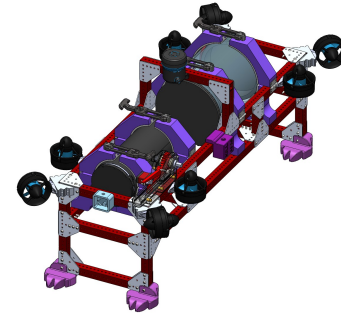


Fig. 1. SeaWolf IX (SWIX) CAD Model

inputs, but its computational intensity is a serious weakness for this use case. As the vehicle must be underwater and untethered, all computation must be done locally and online. Given the size and thermal constraints of the vehicle, its onboard computing is limited to a single Nvidia Jetson, which is also responsible for real-time object detection. As a result, a less computationally intensive controller is needed.

Model Predictive Path Integral Control (MPPI) was first proposed in 2016 [1] and applied to a scaled down rally car driving at the extent of its capabilities. MPPI is an online method that uses a system model and sampling of forward trajectories to approximate the solution to a non-linear MPC problem with significantly less computational effort than the original algorithm. While our vehicle does not require the same ability to drive aggressively, it would benefit enormously from the ability to efficiently carry out model-based control. This project aims to demonstrate the viability of applying this method of control to our vehicle.

II. METHODS

A. The Vehicle

1) *Overview:* SeaWolf IX (SWIX) (Fig.1) is the latest iteration in NC State AquaPack Robotics' autonomous underwater vehicle fleet. Building upon prior iterations, SWIX features an aluminum frame, two dedicated waterproof hulls

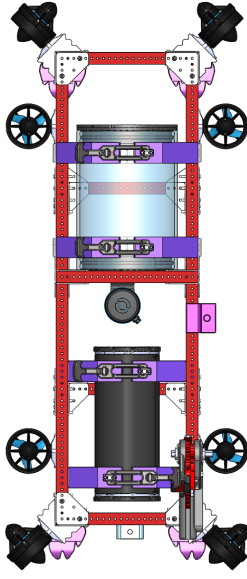


Fig. 2. SWIX CAD Top View



Fig. 3. Simplified model of SWIX for simulation

for electronics and power, and eight Blue Robotics T200 thrusters. The vehicle provides full six-degree-of-freedom actuation, enabling precise maneuvering in constrained underwater environments. Motivated by its advanced configuration and our active involvement in AquaPack Robotics, we apply Model Predictive Path Integral (MPPI) control to a simplified model of SWIX and assess controller performance within a widely used physics-based simulation environment. This simulation serves as a proof of concept and trajectory-tracking testbed, supporting early validation of control strategies prior to potential deployment on the physical platform in future AquaPack Robotics efforts.

2) *Physical Dimensions*: For simulation purposes, SWIX is modeled as a small rectangular prism with a length-to-width-to-height ratio approximating 2:1:1 (Fig. 3). The total length of the simulated vehicle is approximately 0.8 m, with a width and height of 0.4 m. The vehicle is negatively buoyant with a total mass of approximately 22kg. Its center

of gravity and center of buoyancy are both considered to coincide with the geometric center, allowing for simplification of the rigid-body dynamics. This simplified model preserves the primary hydrodynamic characteristics relevant for control and simulation while reducing computational complexity that would come with a more intricate model of the AUV.

3) *Thrust Allocation*: As indicated in II-A1, SWIX is equipped with eight thrusters mounted around the vehicle frame to provide full six-degree-of-freedom actuation. Four vertically oriented thrusters (two on each side) generate forces for pitch and roll control, while four diagonally oriented thrusters contribute to yaw control.

Individual thruster forces are represented as

$$F = [F_0, F_1, F_2, F_3, F_4, F_5, F_6, F_7]^T, \quad (1)$$

where F_i denotes the thrust commanded to the i -th actuator. These forces are mapped to the resultant generalized forces and moments acting on the vehicle through a thrust allocation matrix $B \in \mathbb{R}^{6 \times 8}$:

$$\tau = BF, \quad (2)$$

where $\tau \in \mathbb{R}^6$ is the vector of net forces and torques along the six degrees of freedom:

$$\tau = [T_x, T_y, T_z, \tau_x, \tau_y, \tau_z]^T. \quad (3)$$

Here, T_x , T_y , and T_z denote translational forces in the surge, sway, and heave directions, respectively, while τ_x , τ_y , and τ_z denote rotational moments about the roll, pitch, and yaw axes. Let θ_i represent the mounting angle of each diagonal thruster with respect to the y -axis. Then, the thrust allocation matrix is defined as follows:

$$B = \begin{bmatrix} A_1 & A_2 & 0 & 0 & A_3 & A_4 & 0 & 0 \\ -B_1 & B_2 & 0 & 0 & -B_3 & B_4 & 0 & 0 \\ 0 & 0 & -1 & -1 & 0 & 0 & -1 & -1 \\ 0 & 0 & -l & -l & 0 & 0 & l & l \\ 0 & 0 & l & -l & 0 & 0 & -l & l \\ d & -d & 0 & 0 & -d & d & 0 & 0 \end{bmatrix}, \quad (4)$$

where $A_i = \cos(\theta_i)$, $B_i = \sin(\theta_i)$, l is the moment arm associated with the vertical thrusters, and d is the moment arm associated with the diagonal thrusters, each measured relative to the vehicle's center of mass.

4) *System Dynamics*: The generalized dynamics of the vehicle can be described using the standard manipulator equation [2]:

$$M\dot{v} + C(v)v + D(v)v + g(\eta) = \tau \quad (5)$$

where $M \in \mathbb{R}^{6 \times 6}$ is the mass matrix, $C \in \mathbb{R}^{6 \times 6}$ is the coriolis force matrix, $D \in \mathbb{R}^{6 \times 6}$ is the damping matrix, $g \in \mathbb{R}^{6 \times 6}$ is the vector of restoring forces, $\tau \in \mathbb{R}^6$ is the resultant force, and $v \in \mathbb{R}^6 = [u \ v \ w \ p \ q \ r]^T$ is the velocity in the robot's frame. In the case of our underwater vehicle, this can be further expanded into:

$$M_{RB}\dot{v} + C_{RB}(v)v + C_A(v_w)v_w + D(v_w)v_w + g(\eta) = \tau \quad (6)$$

$$v_w = v - v_c \quad (7)$$

where $M_{RB} \in \mathbb{R}^{6 \times 6}$ is the rigid body mass matrix, $M_A \in \mathbb{R}^{6 \times 6}$ is the added mass matrix, $C_{RB} \in \mathbb{R}^{6 \times 6}$ is the rigid body coriolis force matrix, $C_A \in \mathbb{R}^{6 \times 6}$ is the matrix of coriolis forces due to the added mass matrix, and v_w is the velocity of the water due to current [3].

For the purposes of our simulation, we assumed the current of the water to 0, which causes all of the dependent terms to also be 0. This leaves:

$$M_{RB}\dot{v} + C_{RB}(v)v + g(\eta) = \tau \quad (8)$$

We also assumed that the vehicle was symmetrical about all axes, and that its center of gravity was at its geometric origin, which allows us to simplify the expressions from Wu [3] to obtain:

$$M_{RB} = \begin{bmatrix} m & 0 & 0 & 0 & 0 & 0 \\ 0 & m & 0 & 0 & 0 & 0 \\ 0 & 0 & m & 0 & 0 & 0 \\ 0 & 0 & 0 & I_x & 0 & 0 \\ 0 & 0 & 0 & 0 & I_y & 0 \\ 0 & 0 & 0 & 0 & 0 & I_z \end{bmatrix} \quad (9)$$

$$C_{RB} = \begin{bmatrix} 0 & 0 & 0 & 0 & mw & 0 \\ 0 & 0 & 0 & -mw & 0 & 0 \\ 0 & 0 & 0 & mv & -mu & 0 \\ 0 & mw & -mv & 0 & I_z r & -I_y q \\ -mw & 0 & -mu & -I_z r & 0 & I_x p \\ mv & -mu & 0 & I_y q & -I_x p & 0 \end{bmatrix} \quad (10)$$

By assuming that the center of gravity and the center of buoyancy of the robot are both equal to the geometric center of the robot:

$$g(\eta) = \begin{bmatrix} (W - B)\sin(\theta) \\ -(W - B)\cos(\theta)\sin(\phi) \\ -(W - B)\cos(\theta)\cos(\phi) \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (11)$$

where $W = mg$ (g being the acceleration due to gravity), and $B = \rho g \nabla$, where ρ is the density of water, and ∇ is the robot's volume. To update the state of the vehicle, the following relationship is used [2] [3]:

$$\dot{\eta} = J(\eta)v \quad (12)$$

where $\dot{\eta} \in \mathbb{R}^6$ is the velocity of the robot in the world frame, and $J(\eta) \in \mathbb{R}^{7 \times 7}$ is the mapping from velocity in the body frame to velocity in the world frame.

B. Model Predictive Path Integral Control

1) *Overview:* Model Predictive Path Integral Control (MPPI) is a sampling-based solution to the non-linear MPC problem. The problem we aim to solve can be broadly stated as: find the optimal sequence of control inputs to track a trajectory. Stated another way, find the sequence of control

inputs that minimizes the cost of following a trajectory. Mathematically:

$$u_{optimal} = \min_{u(\cdot), u \in \mathcal{U}} \int_T c(x(t), u(t)) dt \quad (13)$$

However, this is not a convenient problem to solve directly. MPPI aims to approximate the solution to the MPC problem without having to evaluate the integral. The key steps of the algorithm are as follows:

- 1) Randomly sample K sets of T sequential control inputs
- 2) Propagate the vehicle dynamics T steps into the future to generate K paths
- 3) Evaluate each path point-wise according to a predefined cost function
- 4) Weight each control sequence by its respective path's cost function
- 5) Produce a control sequence that is the weighted average of the generated control sequences
- 6) Execute the first control action of the weighted sequence

2) *Formal Algorithm:* The following is an abridged derivation of the controller originally proposed in [1]. We begin by randomly generating $k \in \{1, \dots, K\}$ sequences of $t \in \{0, \dots, T-1\}$ control inputs $u \in \mathbb{R}^m$ to produce $u_k(\cdot)$. For each control parameter $i \in \{1, \dots, m\}$

$$u_{k,t,i} \sim \mathcal{N}(\mu_i, \sigma_i) \quad (14)$$

This means that each control parameter has mean μ_i and standard deviation σ_i which define the sampling distribution from which these controls are sampled from $\forall k, t$. Next, the state $x \in \mathbb{R}^n$ is set

$$x_{k,0} = x_{init}, x_{init} \in \mathbb{R}^n \quad (15)$$

The state x is propagated forward in time with the system's forward dynamics $F : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$.

$$x_{k,t+1} = F(x_{k,t}, u_{k,t}) \quad (16)$$

A cost for each control sequence $u_k(\cdot)$ is then calculated as

$$S(u_k(\cdot)) = \left[\sum_{t=0}^{T-2} c(x_{k,t}, u_{k,t}) \right] + \phi(x_{k,T-1}) \quad (17)$$

$$c(x, u) = x^T Q x + u^T R u, c : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R} \quad (18)$$

$$\phi(x) = x^T P x, \phi : \mathbb{R}^n \rightarrow \mathbb{R} \quad (19)$$

where $Q \in \mathbb{R}^{n \times n}$, $P \in \mathbb{R}^{n \times n}$, and $R \in \mathbb{R}^{m \times m}$, are diagonal matrices with their entries corresponding to cost of the state or input of their respective index.

Each sequence $u_k(\cdot)$ is assigned a weight corresponding to

$$W(u_k(\cdot)) = e^{\frac{-1}{\lambda} S(u_k(\cdot))} \quad (20)$$

where $\lambda \in \mathbb{R}$ is a tuning parameter. Higher values of λ correspond to more exploration of sampled inputs, and a lower value corresponds to less. These weights are then normalized.

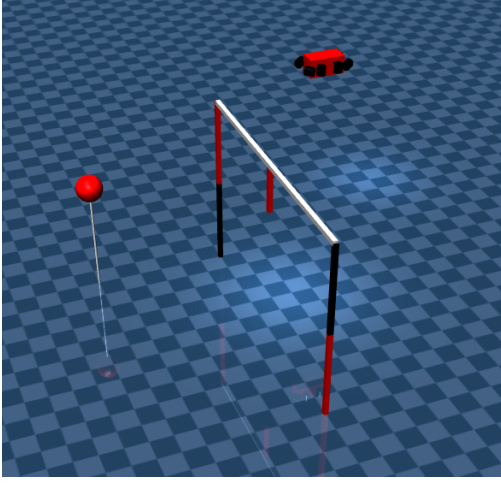


Fig. 4. From left to right: "Buoy", "Gate", Robot

$$W_{normalized}(u_k(\cdot)) = \frac{e^{-\frac{1}{\lambda}S(u_k(\cdot))}}{\sum_{k=1}^K e^{-\frac{1}{\lambda}S(u_k(\cdot))}} \quad (21)$$

The optimal control sequence $u_{optimal}(\cdot)$ is then calculated as the weighted sum of each sequence $u_k(\cdot)$ and its respective normalized weight.

$$u_{optimal}(\cdot) = \sum_{k=1}^K u_k(\cdot) W_{normalized}(u_k(\cdot)) \quad (22)$$

Finally, the first element of $u_{optimal}(\cdot)$ is applied to the system.

$$u^* = u_{optimal,0} \quad (23)$$

C. Physics Simulation

While there are many physics simulation engines that are popular in the robotics community, very few of them specialize in accurately representing the behavior of rigid bodies underwater. While its documentation explicitly states that accurate fluid simulation is beyond its scope [4], we found that MuJoCo's fluid simulation capabilities were developed enough to support our experiments. The software supports two fluid models, the inertial based model and the ellipsoid based model. We used the inertial model, as it was the simpler of the two. In order to enable this behavior, we manually set the viscosity and density parameters of the simulation to match that of water (1000 and $1e^{-6}$ respectively) [5]. Obstacles were added to model the areas of the official RoboSub competition course in which the robot is required to navigate through. These include a "gate" which the robot must travel through, and a suspended "buoy" that the vehicle is required to circumnavigate.

While our derived vehicle dynamics are based on well-established research and the physical parameters of the robot, we found that they did not always match what was propagated forward by the simulation. For the purposes of our testing, we switched from using our derived vehicle dynamics to

headless instance of the physics simulation to propagate our state forward in time for MPPI sampling.

D. Software Implementation

The software for this project was written using Python 3.10. While certainly not the fastest language available, it provides adequate tooling and is simple to use. For simulation and computation, the software relied heavily on the Numpy and MuJoCo libraries.

1) *Trajectories*: Trajectories for the vehicle are prespecified by a series of waypoints. These waypoints each contain a desired vehicle configuration in the global frame, as well as the time after the start of the trajectory that the vehicle is expected to reach this configuration. While velocities are not explicitly stated, they are encoded implicitly due to the fact that both time and position are specified. To reduce the storage requirements of the program, only these waypoints are stored. When other parts of the program consult the trajectory for the desired configuration at a specific timestep, the configuration is derived from a linear interpolation between the closest historical and future specified waypoints.

2) *Dynamics*: Initially, the forward dynamics used for control input calculation came from the model explained in II-A4. While this is no longer the case, these dynamics are fully implemented in code and could be used if they better matched the simulator's dynamics. They were also implemented in such a way that warrants discussion. Though MPPI is in general more computationally efficient than MPC, it can still be too slow for real time use, especially when a higher time horizon or a larger number of rollouts are used. As mentioned previously, the use of Python does not help with the issue of execution time. As a result, extra care must be taken to ensure that the controller is able to produce an effective control input in a reasonable amount of time. To accomplish this, we leveraged the "Numba" library, which compiles targeted functions to machine code prior to program execution. This provides an enormous performance boost, and would potentially enable this controller to be used in real time.

However, as alluded to in II-C a headless instance of the MuJoCo simulator was ultimately used to propagate dynamics forward for control input calculation. This is because our model poorly reflected the true dynamics of the vehicle in the simulator. While this change improved the efficacy of our controller, it came with a significant performance cost. Due to factors that are beyond the scope of this discussion, this change precluded the use of the JIT compilation technique that previously boosted performance.

E. Controller Tuning

The tuning of the controller was done mostly empirically, but occupied enough development time to warrant discussion. We began by finding the amount of thrust needed in the vertical direction to maintain depth. This was used as the mean of the normal distribution from which Z control inputs were drawn during sampling. Next we attempted to track trajectories in which only Z varied and used the results to update the variance

and cost associated with the Z direction. We did the same with each of the other directions, except that the mean of their sampling distributions were 0, as it took no constant controller bias to keep them in place.

Other parameters that had to be tuned were K , T , and λ . We started with a very high value for K , and progressively lowered it after tuning other values to find the lowest value that provided acceptable performance. T was tuned similarly. One thing we noted was that T had to be higher than we expected, because we were controlling force inputs instead of just velocities. This meant that it took longer for control actions to meaningfully improve the tracking error, and the controller therefore needed a longer horizon. To tune λ , we simply picked a small number. This reduces exploration and emphasizes on the best performing control inputs in the final weighting.

III. RESULTS

A. Experimental Setup

To test the performance of the controller, we created a simulated environment in MuJoCo that models obstacles similar to the obstacles the robot is supposed to navigate around at competition. The first is the "gate", a wide vertical beam supported by two vertical beams that extend to the bottom of the pool. The second obstacle is the "buoy", a sphere suspended floating in the water, moored down by a thin pole (fig 4). A trajectory was generated to guide the robot downwards, to pass through the gate, and to then circumnavigate the buoy while keeping the front of the robot pointed towards it. We ran the simulator with $dt=0.0001s$, $K=800$, $T=50$, and a controller update rate of 10Hz. As the simulation ran we sampled the desired and true configuration of the robot also at 10Hz, the results of which are displayed in Fig. 6.

B. Experimental Outcome

As can be seen in Fig. 6, the controller tracked the reference trajectory quite effectively (segment shown in Fig. 5). First, let us address the roll and pitch; for the purposes of this experiment, we simply set the desired values of these properties to be zero throughout the trajectory, and did not sample any control inputs aside from zero to influence them. Because this system is fully actuated, it is unsurprising that the other inputs had minimal influences on those values. The target yaw did, however, have non-zero setpoints, as varying yaw is what allowed to the robot to "point towards" the buoy as it circled the obstacle. The yaw tracking was nearly flawless, with some minor oscillations near sharp changes. These imperfections are expected, as the generated trajectory was not continuous. Because the robot is a physical object and has inertia, the desired trajectory was not completely realizable.

The X, Y, and Z positions also tracked very well, as indicated by the fact that the blue and orange lines are nearly overlapping. Again, the tracking is not perfect for several reasons. One is the aforementioned discontinuities in the trajectory. The other sources of error are the result of known properties of the controller. Firstly, it was run with relatively low values of K

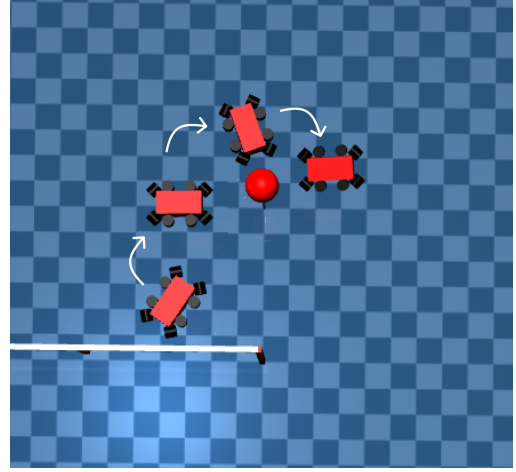


Fig. 5. Robot orbiting obstacle

and T , meaning that it evaluated a low number of trajectories with a limited view of the future. Due to the low horizon, the controller was not necessarily able to anticipate sharp changes with enough time to counter the inertia of the vehicle. The limited number of samples simply decreases the probability that the controller is able to find the truly "optimal" solution. For the sake of discussion, consider the controller with $K=1$. The probability that this set of rollouts contains desired trajectory is effectively zero. As K approaches infinity, the probability that the set of rollouts contains the optimal trajectory approaches 1, meaning that the controller has found a control sequence to exactly produce the optimal trajectory. For any finite K , the likelihood varies with K , with small values generally producing sub-optimal control sequences, and larger values producing better sequences. Because $K=800$ is a fairly low number for 6 DOF, it is expected that the outputted control sequence will be sub-optimal. Considering this observation, the tracking performance is quite impressive.

IV. DISCUSSION

A. Design Choices

As stated in the introduction and throughout this report, we chose to explore the use of MPPI for our underwater vehicle. We predicted that using a model-based controller would be ideal for an environment with well-understood dynamics, such as water. However, traditional model-based controllers can be computationally intensive and would be challenging to implement given the hardware restrictions of our physical vehicle. We also chose to heavily simplify the original dynamics of the vehicle, as many parameters would otherwise have to be obtained through a rigorous system identification routine [3], which would produce different values in simulation versus in practice anyway. Currents in the water were also neglected for simplicity. In terms of trajectory generation, we did not ensure desired trajectories were continuous, also for simplicity. This is because the only critical configurations were the ones specified in the waypoints, and the obstacle field was sparse

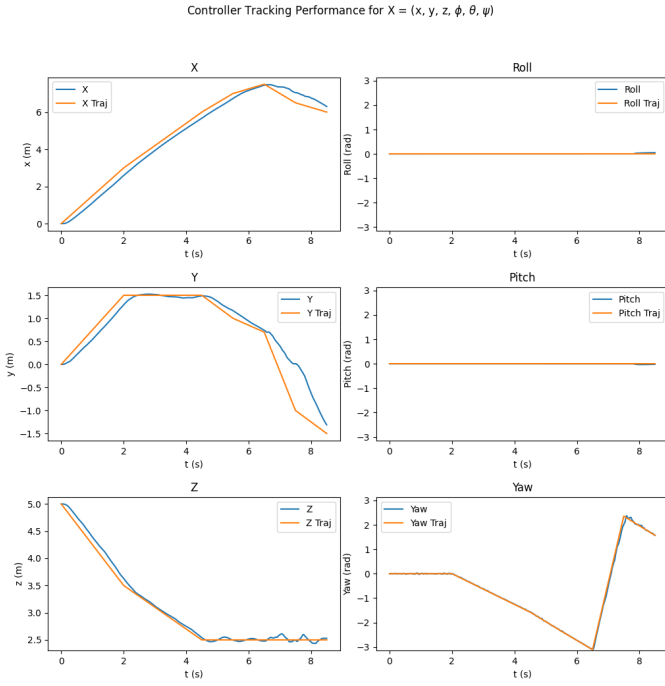


Fig. 6. Robot Trajectory vs Followed Path

enough that slight deviations while traveling from waypoint to waypoint would not severely negatively affect the vehicle.

B. Design Challenges

Throughout this project, we encountered many design challenges. The first, encountered in the initial planning phase, was that there is a striking lack of physics simulation platforms capable of producing accurate underwater simulations. While we eventually found that MuJoCo had limited support, finding a platform on which to conduct our experiments was not a small task.

The first major challenge we encountered in implementation was the discrepancy in our calculated dynamics and the simulator dynamics, detailed in II-D2. For the sake of this prototype, we solved this problem by using another instance of the MuJoCo physics simulator for propagating our system forward in time. While this greatly increased the controller's ability to track the reference trajectory, it severely negatively affected the required computation time. For our purposes, this simply meant that it took longer to run simulations. An alternative solution will have to be found when implementing this on physical hardware, which will likely involve using a compiled language and fine-tuning our dynamical model.

Tuning was another large challenge, and was described in detail in II-E. The biggest issue when initially tuning the controller was that it was not clear when poor tracking was a result of a bug in the controller or poor tuning. It is difficult to verify that that controller is functionally correct when the controller values are not tuned, and it is difficult to tune the controller values when the controller has not been verified to work. Another difficulty here is the fact that there are many

knobs that can be turned: K , T , λ , Q , R , and P , as well as the means and variances of the sampling distributions for each control input. Our solutions to these issues were multi-modal, and included physical reasoning, isolating individual degrees of freedom, logging internal controller variables, and trial and error.

V. CONCLUSION

In this project, we implemented an MPPI controller on a 6 DOF Autonomous Underwater Vehicle to track a predetermined trajectory. We demonstrated that this controller was able to accurately track the trajectory's X, Y, Z, and yaw reasonably well, even with low values of K and T and discontinuous trajectories. While the tracking was not perfect, it was good enough for expending minimal computational effort and in a sparse obstacle field.

Despite this success, there are multiple areas for further improvement. The first is removing the dependency on the MuJoCo simulator for forward dynamics, as that likely only represents our dynamics accurately within the simulator and is computationally inefficient. Better would be to employ the full dynamical model in [2], and perform the system identification necessary to represent the robot's behavior accurately. This would allow the controller to be implemented in real time on physical hardware. Another potential improvement would involve dynamically generating trajectories based on perception data from the robot's sensors. The controller could also incorporate constructs such as Control Barrier Functions to ensure safety in the water. Regardless of the improvements still to be made, we believe that this project represents a leap in forward in terms of applying modern control techniques to an understudied field. We hope that this will assist us in our goal of developing precise, dynamic underwater locomotion.

REFERENCES

- [1] Grady Williams, Paul Drews, Brian Goldfain, James M Rehg, and Evangelos A Theodorou. "Aggressive driving with model predictive path integral control," In Robotics and Automation (ICRA), 2016 IEEE International Conference on, pages 1433–1440. IEEE, 2016.
- [2] T. I. Fossen and O.-E. Fjellstad, "Nonlinear modelling of marine vehicles in 6 degrees of freedom," *Journal of Mathematical Modelling of Systems*, vol. 1, no. 1, pp. 1–?, 1995.
- [3] C.-J. Wu, "6-DoF modelling and control of a remotely operated vehicle," Ph.D. dissertation, College of Science and Engineering, Flinders University, Adelaide, Australia, 2018.
- [4] DeepMind, "MuJoCo Computation: Fluid Forces," MuJoCo Documentation. [Online]. Available: <https://mujoco.readthedocs.io/en/stable/computation/fluid.html>.
- [5] DeepMind, "MuJoCo XML Reference: body/light light intensity," MuJoCo Documentation. [Online]. Available: <https://mujoco.readthedocs.io/en/stable/XMLreference.html#body-light-intensity>.